NoSQL platforms and distributed data stores

dr hab. inż. Maciej Grzenda, prof. uczelni
M.Grzenda@mini.pw.edu.pl
http://www.mini.pw.edu.pl/~grzendam

Politechnika Warszawska
Wydział Matematyki i Nauk Informacyjnych

Warsaw University of Technology

MSc program in Data Science has been developed
as a part of task 10 of the project
„NERW PW. Science - Education - Development - Cooperation”
co-funded by European Union from European Social Fund.

Objectives

- The need to store Big Data resources raised the interest in non-relational database-like platforms
- Hence, NoSQL platforms have been developed
- These platforms do not aim to replace relational databases
- NoSQL platforms address new use cases such as:
  - storing large data resources, while still being able to perform random read at least i.e. find and read small data entities efficiently (which is not possible with Hadoop only)
  - storing graph data
  - storing data that has no fixed, carefully designed data model
- The primary objective of this lecture is to show key aspects of NoSQL platforms

Resources

1. [ALAPATI2018] Alapati, Sam R, Expert Apache Cassandra Administration, Apress, Berkley, 2018
2. [MARDAN2018] Mardan, A., Full Stack JavaScript. Learn Backbone.js, Node.js, and MongoDB, Apress, 2018 (Full Stack Java Script : poznaj technologie Backbone.js, Node.js i MongoDB, Helion, 2020)
3. [NOSQL] P. Sadalage, M. Fowler, NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence, Addison Wesley Professional, 2012
4. [GROVER2015], Mark Grover, Ted Malaska, Jonathan Seidman, and Gwen Shapira. Hadoop Application Architectures. O'Reilly, 2015
5. [NOSQLWEB] https://hostingdata.co.uk/nosql-database/

NoSQL?

- The first influential use of the term is attributed to Johan Oskarsson, who selected NoSQL as a Twitter hashtag for a meeting on new DBMS in 2009,
- The use of the term has been spreading out since that time, with no clear definition,
- This can be treated as a symbol of changes from:
  - the times of relational databases (mathematical theory developed by E. Codd)
  - to the times of NoSQL (open source/Twitter/Facebook etc. movement with no clear definition of NoSQL at all)

NoSQL clarified (to a possible extent)

When attempts to define NoSQL term are made, most frequently the following features are highlighted:

- Virtually all NoSQL DBMSs are non-relational
- Most of the time NoSQL DBMSs do not support SQL or SQL is not the primary way of manipulating the data
- Most of NoSQL DBMSs are open source products
- Most of NoSQL DBMSs run on clusters
- The term is usually reserved for systems developed in 21st century i.e. following the explosion of data processing needs of web systems

1. **Even for these elementary features, contradictory opinions can be found**
2. **Due to all these ambiguities, [NOSQLWEB] proposes this definition of NoSQL: *" Next Generation Database Management Systems mostly addressing some of the points: being non-relational, distributed, open-source and horizontally scalable."***
3. **Another"definition": *"an ill-defined set of mostly open-source databases, mostly developed in early 21st century, and mostly not using SQL"* [NOSQL]**

NoSQL = No SQL?

- [NOSQL] suggests that NoSQL databases do not use SQL
  - not really true as SQL support (at least partial) was added e.g. by Cloudera to Apache HBase
- NoSQL?
  - *the community now translates it mostly with "**not only sql**"* [NOSQLWEB]
- Not only SQL is the most frequent (and more accurate) explanation of NoSQL

NoSQL – no fixed schema

- Usually operates without a schema i.e. fields can be added to individual records with no need for prior changes to database structure
- Addresses one of the limitations of relational systems – i.e. the need for more flexible data model
- Frequently, in NoSQL there is no single data model answering the data needs. The actual model may depend on envisaged usage of the data and may include denormalisation and redundancy such as describing many-to-many A-B relation at both entity A and B (which is redundant, but may help improve read performance)

Let us imagine an asset inventory system, keeping the data of 30 types of assets. This means in RDBMS two basic options:

1. **30 tables matching the needs of individual types. What if 4 more types have to be added? 4 more tables? What if the clients want to be able to define new types in themselves?**
2. **1 table with columns like NumberFeature1 used for keeping e.g. diameter for a pipe and RAM size for a desktop computer.**

Column-family databases

- Keep data of K-th row as: (key_K, columnValue(K,1), ColumnValue(K,2),…,columnValue(K,N_k))
- The set of used columns can be different in each row
- The number of specified columns N_k can be also different in every row
- The columns are grouped into column families,
- Column families are groups frequently accessed together (e.g. for a customer: all address columns, all tax-related data, but also: all orders, with each column representing one order)

Sample structure – orders table

Column family: order data | Column family: ordered products

Key	OrderDate	DeliveryDate	Status	Product101	Product20	Product42
1045	10/02/2014	15/04/2012	DELIVERED	45.0	22	17.54

Column family: order data | Column family: ordered products

Key	OrderDate	CancelDate	Status	Product78
1048	11/02/2014	11/02/2014	CANCELLED	22#black

Each row may use its own set of columns. Each column may contain atomic value, but also composite value (e.g. Product#78). Each column may host different data types. This kind of a table can be created in Apache HBase

Column-family databases – any rules?

- Still, there are some rules:
  - One column belongs to exactly one column family
  - Column families are rarely added; this may even require stopping the database
- Comments:
  - Column family can resemble a table. Many column families in one table may mean aggregating the data from many tables in terms of relational model (here: Orders and Order Details tables from relational model are reflected by two column families) to one NoSQL table
  - In Cassandra there are no column families per se, but there are supercolumns i.e. columns with nested columns

NoSQL examples

Data model	Sample DBMS
Column store/ Column families	Hadoop/HBase Apache Cassandra
Document	MongoDB RavenDB
Key-value	Redis Riak BerkeleyDB
Graph	Neo4j HyperGraphDB
XML	EMC Documentum xDB BerkeleyDB XML

[NOSQLWEB] lists **225+** NoSQL DBMS

Aggregates?

	Aggregate-ignorant DB	Aggregate-based DB
Idea	The unit of interaction is not defined. Instead there are many interrelated atomic rows	A unit of interaction with the data can be defined. This can be an order, an invoice, etc.
Positive consequences	1. Depending on the needs, the same data can be accessed in variety of ways e.g. starting from most frequently ordered products or recent orders. 2. No need to identify aggregates, which could be difficult or application-dependent (is it better to have one aggregate: customer with embedded orders or two separate aggregates: customer and order?) 3. Better when no single primary structure for manipulating the data exists i.e. there is no single dominating aggregate, the data is manipulated with	1. The knowledge of aggregate can be used to store and distribute data e.g. to put entire data of an order on the same host in a cluster (sharding!) 2. It helps to put the data on the cluster. DBMS knows which data should be placed on the same node. Simply, all data of one row being an aggregate is stored on one cluster node.
Sample systems	Relational DBMSs, NoSQL graph databases	Key-value, column-based, document NoSQL databases

Aggregates and ACID

	Aggregate-ignorant DB	Aggregate-based DB
Idea	The unit of interaction is not defined. Instead there are many interrelated atomic rows.	A unit of interaction with the data (i.e. the aggregate) can be defined
ACID Transaction	In OLTP relational DBMS can span any number of records from any number of tables	1. The data from one aggregate is modified under ACID transaction rules. 2. When multiple aggregates are supposed to be modified within one transaction, this has to be handled by a client application i.e. is not supported by DBMS directly.
Comments	Little or no need to consider transactions when designing data structure i.e. database tables.	The need for transactions should be taken into account when designing aggregates i.e. data structures

Is there really no schema in schemaless database?

- In a key-value store, but also document and column-family oriented data store it is possible to store any data under a key
- Still, the client application
  - will most likely have to assume some data structure
  - for instance it will assume that it is the column NetIncome that contains a number with net income or that this data is stored at a certain place in an XML document.
- Thus:
  - while the schema is not a priori reported to DBMS (hence there is no *explicit schema*), *implicit schema* exists.
  - implicit schema is the schema expected by a client application.

Schemaless problems

- Implicit schema can be a problem if not managed appropriately.
  - In fact, it assumes the consistent behaviour of all data writers and readers.
  - Otherwise, chaotic data can be easily created with significant overhead of data cleaning operations following it.
- The management of implicit schema becomes even more problematic when multiple applications interact with the same data.
- One of the ways of addressing the problem is to encapsulate data logic and integrate it with other applications via web services.

Dealing with cross-aggregate queries

- Aggregate-ignorant DBMS make it relatively straightforward to execute queries referring to multiple rows in multiple tables
- This includes relational DBMS
- Aggregate-based DBMSs may require the use of materialised views i.e. precomputed results for typical queries. These views may be updated
  - In an on-line manner i.e. when source data changes
  - Periodically or on request

Aggregate-based and aggregate-ignorant examples: MongoDB

Basic facts about MongoDB

- MongoDB is among highest-ranked non-relational database management systems and NoSQL platforms
- MongoDB is an example of document-based system
- Data in MongoDB are stored as JSON-based documents
- Even though MongoDB is a document-based platform, it supports indexing, queries and data aggregation
- Further reading: https://docs.mongodb.com/manual

Note that further details about MongoDB are provided in a separate presentation, which contains extended coverage of the platform. More detailed discussion of MongoDB is a part of the module on data storage in Big Data platforms, a part of BSc in Data Science program

Aggregate-based collection

- Let us create a simplified aggregate-based collection of Orders

```
db.MyDenormalisedOrders.insertOne({
  "_id":1016,
  "customer":{
    "id":17,    "name":"ABC Company", "country":"Poland",  "city":"Warszawa","address":"Pl. Politechniki 17"},
  "value":1000,
  "details":[
    {
      "productId":15, "quantity":7
    },
    {
      "productId":22, "quantity":47
    },
    {
      "productId":45,          ity":22
    }
  ]
})
```

Embedded customer data

Embedded order details data

This illustrates how data is stored in a MongoDB collection of documents. In this case, one document contains embedded data placed in many relational tables. Hence, data are denormalised and no longer take the form of records.

Aggregate-ignorant collections

- Let us create a simplified aggregate-ignorant table of Orders and Order Details and place some data in it

```
>db.MyNormalisedOrders.insertOne({
  "_id":1016,  "customerId":17,  "value":1000 })
>db.MyNormalisedOrders.insertOne({
  "_id":1017,  "customerId":17, "value":1000 })
```

```
>db.MyNormalisedOrderDetails.insertOne({
  "orderId":1016,  "productId":15, "quantity":7
})
{   "acknowledged" : true,
    "insertedId" : ObjectId("5ff6cc24b5128ed095f99205")
}
```

```
>db.MyNormalisedOrderDetails.insertOne({
  "orderId":1016,  "productId":22, "quantity":47})
>db.MyNormalisedOrderDetails.insertOne({
  "orderId":1016,  "productId":45, "quantity":22})
```

Linking aggregate-ignorant collections

- Let us link MyNormalisedOrders and MyNormalisedOrderDetails

```
db.MyNormalisedOrders.aggregate([{
    $lookup:
     {
       from: "MyNormalisedOrderDetails",
       localField: "_id",
       foreignField: "orderId",
       as: "orderedProducts"
     }
 }])
```

Note that SQL is not supported. Since, data are not stored in relational way, queries have to be made using API

This is the name of the table to be placed in the output of the query

- The output of the query contains embedded linked data

```
{ "_id" : 1016, "customerId" : 17, "value" : 1000, "orderedProducts" : [ { "_id" :
ObjectId("5ff6cc24b5128ed095f99205"), "orderId" : 1016, "productId" : 15, "quantity" : 7 }, { "_id" :
ObjectId("5ff6cdd1b5128ed095f99206"), "orderId" : 1016, "productId" : 22, "quantity" : 47 }, { "_id" :
ObjectId("5ff6d620b5128ed095f99207"), "orderId" : 1016, "productId" : 45, "quantity" : 22 } ] }
{ "_id" : 1017, "customerId" : 17, "value" : 1000, "orderedProducts" : [ ] }
```

MongoDB can work with both aggregates and aggregate-ignorant data models. This is a unique feature. In most NoSQL platforms there is no way to join tables.

Data storage in Apache Cassandra

Basic facts about Apache Cassandra

- Apache Cassandra (or simply Cassandra) is among highest-ranked non-relational database management systems and NoSQL platforms
- While MongoDB is an example of a document-based system, Apache Cassandra relies on the so called wide-column pattern and is an example of a column family database
- Cassandra supports queries in CQL, which can be considered a simplified SQL. This results in shorter learning curve than in the case of some of the alternative platforms
- Cassandra relies on a masterless architecture. It is highly scalable and highly resilient to failures

Note that further details about Apache Cassandra are provided in a separate presentation, which provides extended coverage of the platform. More detailed discussion of Apache Cassandra is a part of the module on data storage in Big Data platforms, a part of BSc in Data Science program

Physical organisation of data in Apache Cassandra

- **Cassandra cluster**:
  - Can be a single-node cluster or a multi-node cluster
  - Can use a single datacenter or many datacenters
  - Is referred sometimes to as a **ring**, as all nodes in a cluster are equal
- **Datacenter:**
  - Is a multi-node cluster
  - Has a unique name e.g. mycenter
  - Can be composed of many racks
  - Can have its own replication factor
  - Must never span many physical locations
- **Rack:**
  - Contain nodes. The data from a node in the rack is replicated to other racks, if possible.
  - Has a name e.g. rack1
  - May correspond to a physical rack

Physical organisation of data in Apache Cassandra

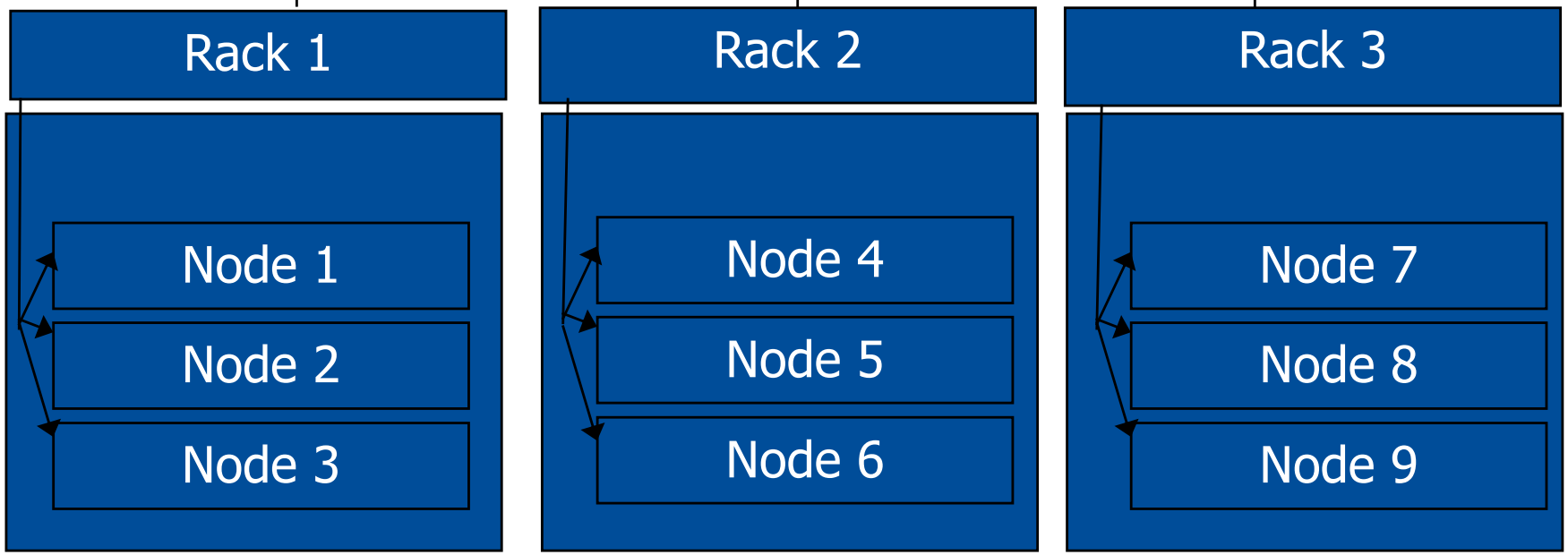
- A node:
  - Can be a server or a VM
  - All nodes are equal. There is no master node
  - Belongs to one rack
  - Nodes contain partititions i.e. subsets of rows from individual tables
- By creating a cluster with multiple data centers, worldwide clusters replicating data between individual data centers can be deployed
- Moreover, different replication settings for different keyspaces (and tables contained in these keyspaces in turn) can be applied
- This provides an extremely scalable architecture

Already in 2014, Apple reported its Cassandra instance to include more than 75,000 nodes and store more than 10 petabytes of data.
https://www.datastax.com/blog/apache-cassandratm-four-interesting-facts

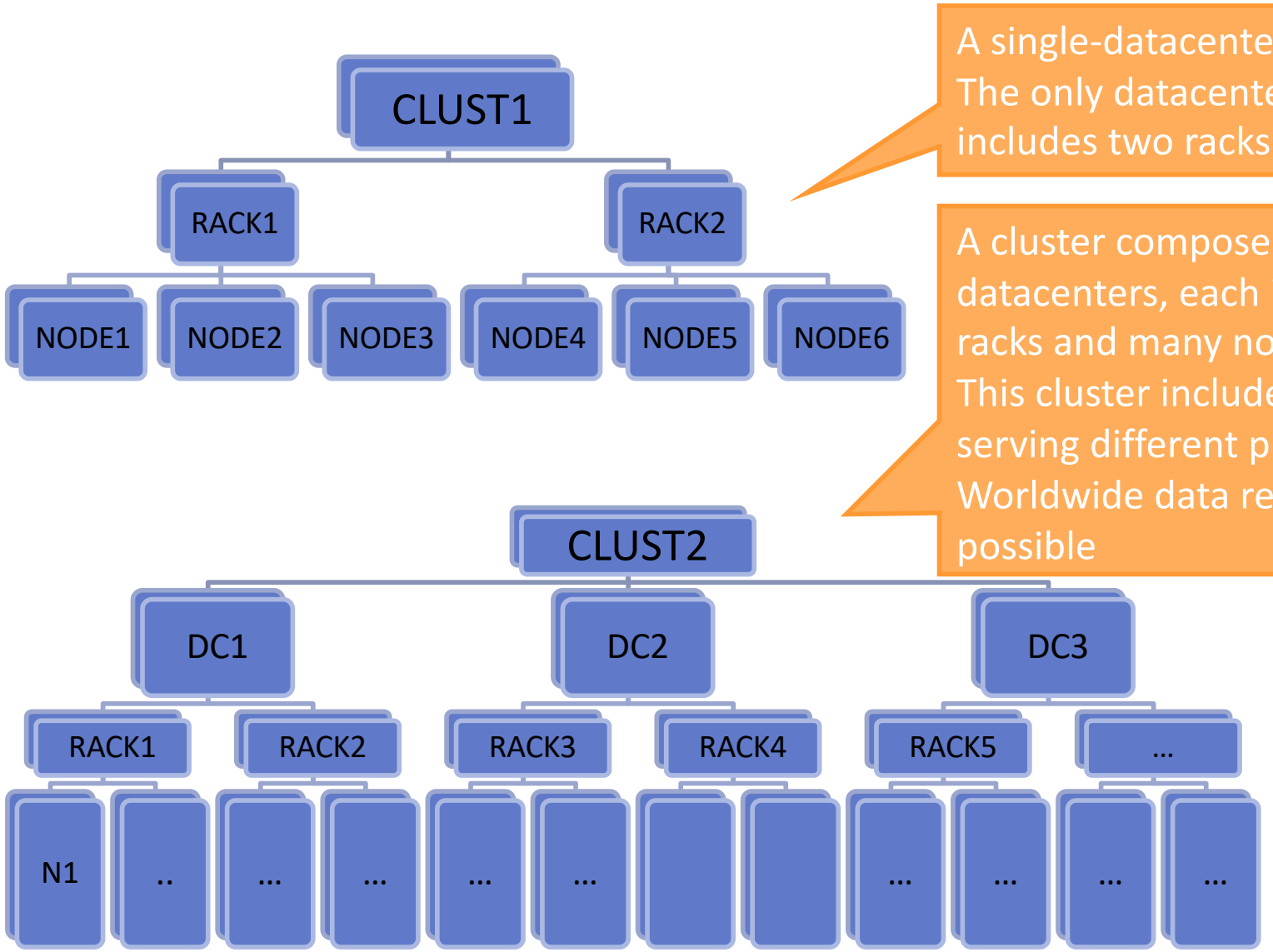
Apache Cassandra architecture: sample cluster

Once racks are defined, Cassandra tries not to replicate data within the same rack. Otherwise, in the case of rack failure, all replicas could be unavailable.

There is no master, but some nodes may play the role of seed nodes i.e. the nodes that help other nodes to learn the topology of their cluster.



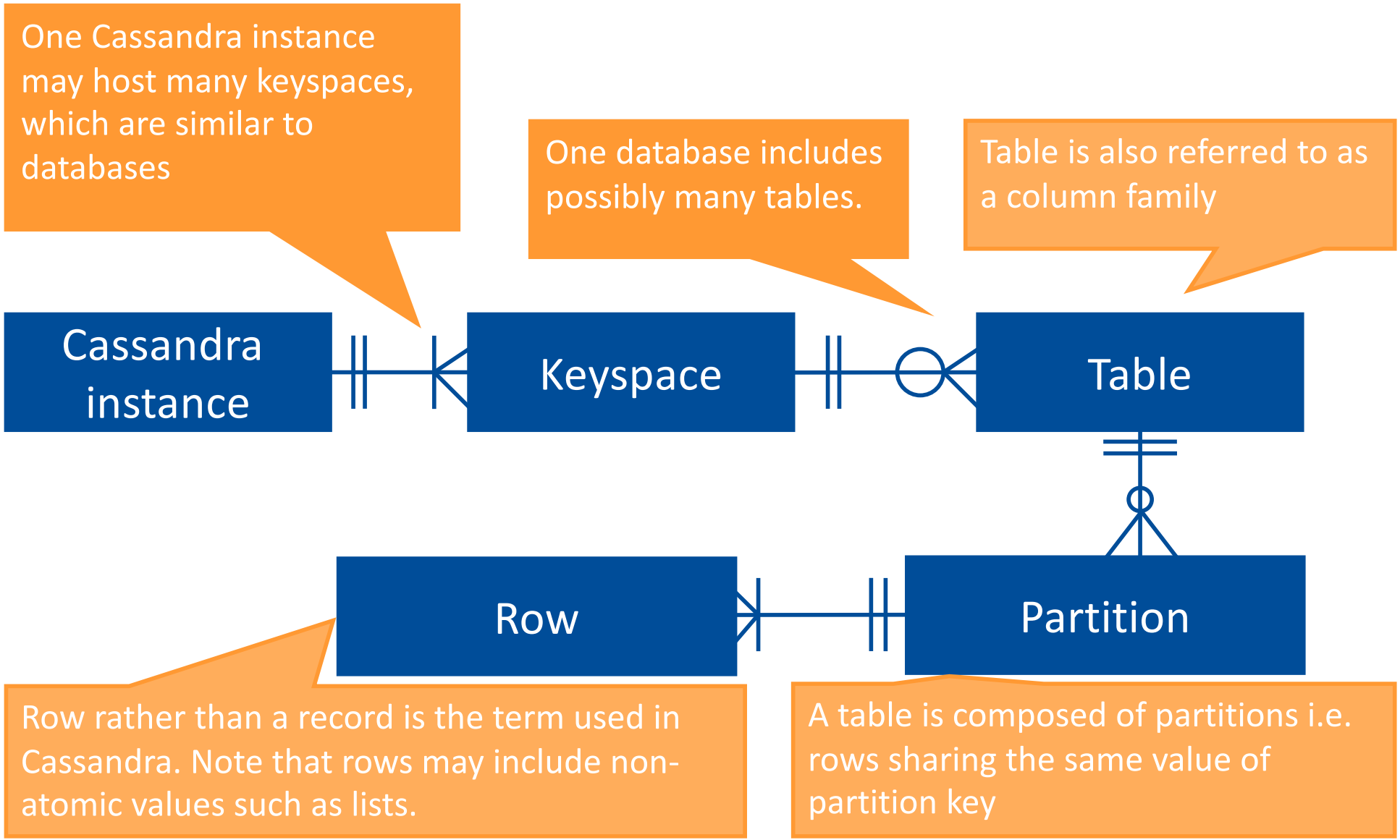
Further cluster configurations



A single-datacenter case of cluster. The only datacenter in this cluster includes two racks

A cluster composed of three datacenters, each including many racks and many nodes.
This cluster includes datacenters serving different physical locations.
Worldwide data replication is possible

Data organisation in Cassandra



Apache Cassandra use cases

- Apache Cassandra can be used to set up multiple data centers and data replication between these data centers
- Moreover, very large tables are supported, as their data can be divided into partitions:
  - stored at different nodes
  - replicated to additional nodes to reduce the risk of limited data availability
- Moreover, CQL query language does not support time consuming operations. As an example, JOIN is not supported. Similarly, only some filtering and sorting operations are possible
- Hence, it can be scaled even to worldwide deployments grouping multiple data centers with many nodes
- This raises the question of consistency of data and a possible conflict with high availability. This question is common for all distributed data stores used to store large volumes of data

Key features of distributed data stores
CAP theorem

Key features of distributed data stores: consistency

- **Consistency** (*Spójność*) in a distributed data store means that a read operation always returns the most recently written data
- **Strict consistency** (*Ścisła spójność*) is not easy to get in a distributed system, as:
  - some of the nodes possibly keeping more recent data may be not available
  - there may be problems with applying some changes to all nodes keeping replicas of affected data
  - hence, strict consistency would mean the risk of not answering some requests due to problems with some cluster nodes, even though other nodes have copies of the data and could serve it and/or update it
- **Eventual consistency** (*Ostateczna spójność*) means that all replicas of the data eventually (i.e. after some time) will become consistent. During that time, read requests may be answered, which means the risk of using not the most recent data for answering these requests.

Note that consistency in this case is not defined in the same way as in ACID

Key features of distributed data stores: Availability and Partition tolerance

- **Availability (***Dostępność***)** means that all clients can get answers for their requests, rather than errors, even though the answer may not contain the data most recently written to a data store
- **Partition tolerance** (*Odporność na partycjonowanie*) – the system works despite possible network connectivity issues
- **CAP theorem** by Eric Brewer states that only two out of three features (**C**onsistency, **A**vailability, **P**artition tolerance) are possible at the same time.
- In practice, as network problems are unavoidable, consistency or availability has to be selected. The problem is both cannot be provided

Further aspects of CAP theorem

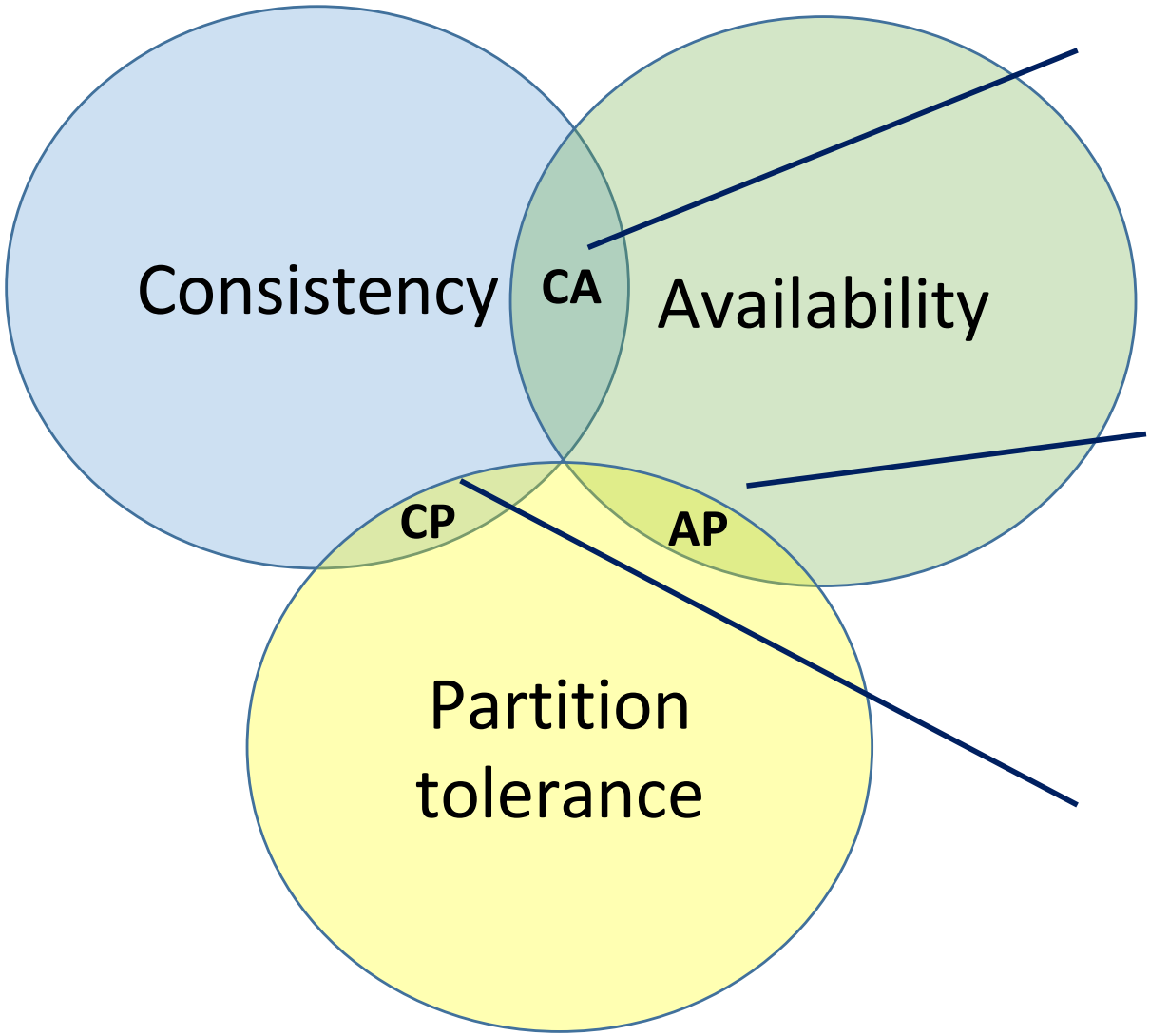
- A more formal phrasing of Brewer's theorem is as follows:

  (..) in a network subject to communication failures, it is impossible for any Web service to implement an atomic read/write shared memory that guarantees a response to every request.

  Source and further details, including the proof of CAP theorem: S. Gilbert, N. Lynch, *Perspectives on the CAP Theorem*, in: Computer, IEEE, vol. 45, no. 2, pp. 30-36, 2012

- Note that there are two typical cases:
  - Consistency is ensured, but only best-effort availability is provided i.e. the systems will be as responsive as possible
  - Availability is ensured, but only best-effort consistency is provided i.e. consistency is sacrificed. This is the right approach when e.g. the application is deployed not just in one data center, but relies on worldwide services relying on web caches. In such cases, strict consistency is not guaranteed, as the impact of failures could be devastating for the service if strict consistency was assumed.

CP and AP distributed databases



In real-life we need partition tolerance. Hence, CA systems are not really useful.

This option means priority given to availability over consistency. Cassandra basically belongs to AP systems. AP systems are also the systems referred to as BASE systems

CP systems prioritise consistency over availability. Cassandra can be configure to behave (almost) like a CP system

NoSQL platforms revisited

- Platforms aiming to provide Big Data storage have to rely on distributed data storage, i.e. typically some NoSQL platform
- Similarly, data stores relying on many geographically distributed clusters rely on distributed data stores and NoSQL platforms
- In such cases, CAP theorem has to be taken into account. In particular, whether consistency or availability is preferred has to be decided.
- Therefore, NoSQL platform has to be selected not just relying on the volume of data, but business needs, which may result in varied:
  - need to execute complex queries and rely on data indexing
  - emphasis on availability vs. consistency
  - need for ACID processing, typically very limited in NoSQL platforms

Relational platforms

vs.

NoSQL platforms including distributed data stores

RDBMS vs. NoSQL platforms

Feature	RDBMS	Apache Hadoop	NoSQL (columnar databases)
Transactions	Extensive support At the level of multiple records and even multiple databases	Not in DBMS way. Hadoop is not a DBMS	Most likely at the level of a single row only
Referential integrity	Can be monitored and enforced by DBMS	A client application is responsible for keeping data consistent	A client application is responsible for keeping data consistent
Scalability	Limited, preferably based on a single high performance server	Virtually unlimited, multiple hosts can form a cluster and share the load of the system	Virtually unlimited, multiple hosts can form a cluster and share the load of the system
SQL support	Extensive support for SQL, yet SQL dialects exist	No SQL at all (unless other projects are used)	No SQL at all or a subset of SQL only
Typically used for	Enterprise resource planning (ERP), financial, accounting systems,…	Batch processing of large volumes of data clickstreams, unstructured content (documents), logs, …	Random access to large volumes of data clickstreams, unstructured content (documents), logs, …

The future of relational databases?

- Will still have a key role, when:
  - Key business data (accounting, financial, orders,…) is processed,
  - Transactional processing spanning multiple operations is mandatory,
  - Many relations in the data exist,
  - Referential integrity is mandatory,
  - Sophisticated SQL queries have to be supported,
  - …
- Relational databases will be the most common form of database in use

Still, the phenomenon of variety of NoSQL databases is strong enough:

1. **not to ignore the fact**
2. **be able to select an appropriate data store for the problem, not necessarily meaning an appropriate relational DBMS**

Apache Hadoop vs. NoSQL

- Apache Hadoop is not a NoSQL platform (or is a NoSQL platform only in the broadest sense):
  - Apache Hadoop:
    - is oriented on batch processing performed on entire, ideally large files
    - does not impose any internal structure and interpretation of the files
  - NoSQL platforms offer the ability to perform random access i.e. access individual rows based on their identifiers and should be used when random access is the dominating use pattern [GROVER2015]
- Still, schema-on-read is the approach common for Apache Hadoop and NoSQL platforms

Graph databases

- Unlike other categories of NoSQL DBMSs, not oriented on clusters
- Allow to store and query graphs i.e. collections of nodes and edges
- Different approaches can be observed:
  - FlockDB – just nodes and edges
  - Neo4j – in addition Java objects can be added as properties of both nodes and edges
- Offer increased performance of search operations, which is largely attained by extra overhead at the time of inserting the data
- Search operations are expected to be mostly (complex) relationship searches
- Most likely the data is kept on a single server
- ACID operations are:
  - mandatory - many nodes and edges have to be updated at the same time to preserve graph structure
  - easier to implement, as most likely everything happens on a single server

Summary

- NoSQL platforms are commonly accepted DBMS platforms
- They do not aim to replace RDBMS platforms
- Still, for some use cases such as processing semi-structured data, and Big Data processing, NoSQL platforms provide the solution
- Hence, knowing when to use (and not use) relational databases, Apache Hadoop and NoSQL platforms is important for both software developers and data scientists nowadays

Warsaw University of Technology

MSc program in Data Science has been developed
as a part of task 10 of the project
„NERW PW. Science - Education - Development - Cooperation”
co-funded by European Union from European Social Fund.